Energy and AI 24 (2026) 100710

Contents lists available at ScienceDirect

Energy and AI

journal homepage: www.elsevier.com/locate/egyai

Agentic framework for programmatic crystal structure generation using a fine-tuned worker–supervisor large language model

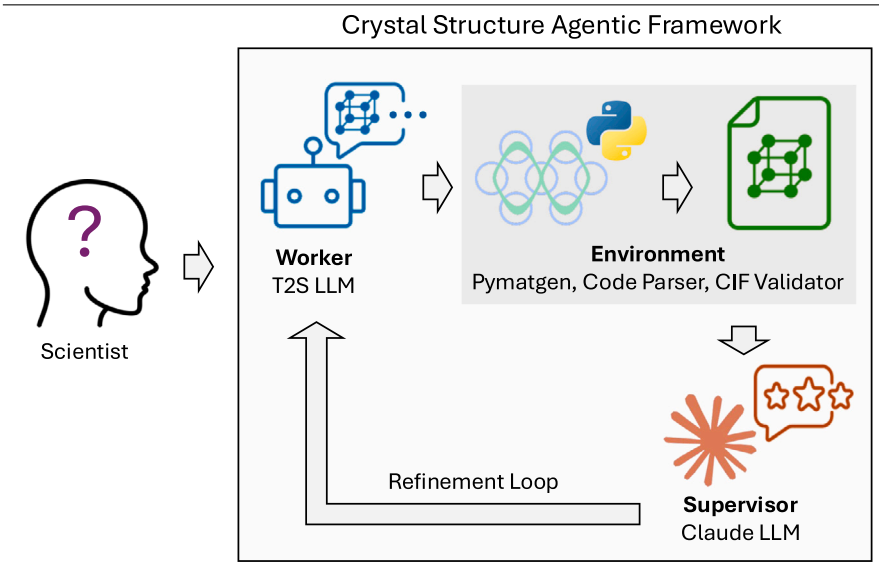
Viktoriia Baibakova *, Alexey Serov

Electrification and Energy Infrastructures Division, Oak Ridge National Laboratory, 1 Bethel Valley Road, Oak Ridge, 37830, TN, USA

HIGHLIGHTS

- *AI agentic framework*: a fine-tuned compact worker and a large supervisor LLMs.
- *Responsible AI*: 350M-parameter worker LLM requiring less training resources.
- *Built-in structure manipulation*: supercell scaling, strain, vacancy, substitution.
- *Performance*: hallucinations decreased from 100% to 5%, with match accuracy up to 82%.
- *Practicality*: applied to benchmark catalysts IrO_2, $Pb_2Ir_2O_7$, Ni_2FeO_4, Ni_3Mo.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
AI agent
Large language models
Responsible AI
Crystal structure
Catalysts

ABSTRACT

Platinum group metals (PGMs) underpin many catalytic technologies but face severe supply constraints, motivating the search for alternative materials and computational methods to accelerate discovery. While atomistic simulation tools such as Pymatgen and ASE have streamlined structure manipulation, they require detailed inputs, limiting accessibility for experimentalists and slowing early-stage exploration. Here, we present an AI-driven agentic framework that orchestrates worker–supervisor large language models (LLMs). The worker translates natural-language prompts of varying abstraction into valid crystallographic structures using a compact LLM fine-tuned with low-rank adaptation on a curated text–code–CIF dataset, emphasizing energy-efficient training. Benchmarking against the baseline CodeGen-350M-mono model shows that fine-tuning reduces hallucination rates from 100% to as low as 5% and improves structural match accuracy to up to 82% for fully specified inputs. Accuracy declines with decreasing prompt detail but remains nontrivial even when only stoichiometry and space group are provided, underscoring the LLM's capacity for crystallographic inference. The supervisor Claude LLM evaluates the outputs and triggers iterative refinement through the worker's built-in structure manipulation capabilities (e.g., supercell scaling, strain, vacancy, and substitution operations). We further demonstrate use cases for technologically relevant catalysts, including IrO_2, pyrochlore

* Corresponding author.
E-mail address: baibakovav@ornl.gov (V. Baibakova).

https://doi.org/10.1016/j.egyai.2026.100710
Received 3 September 2025; Received in revised form 20 February 2026; Accepted 26 February 2026
Available online 2 March 2026
2666-5468/© 2026 Oak Ridge National Laboratory. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

$Pb_2Ir_2O_7$, Ni_2FeO_4, and Ni_3Mo, where the framework generates physically consistent structures that can be refined via geometry optimization. This work introduces a low-energy, language-driven pathway for integrating human and machine intelligence in materials design, paving the way for AI-assisted synthesis planning and high-throughput screening of complex oxides.

1. Introduction

Platinum group metals (PGMs) — platinum (Pt), palladium (Pd), rhodium (Rh), ruthenium (Ru), iridium (Ir), and osmium (Os) — are highly valued for their exceptional corrosion resistance, high melting points, and catalytic activity, making them indispensable in industries ranging from automotive and medical to robust energy technologies such as fuel cells [1–3]. In catalysis and hydrogen generation, multicomponent oxides [4] incorporating PGMs play a particularly critical role, with noble metals such as Pt, Rh, and Ru representing the state-of-the-art catalysts [5,6]. However, PGMs are among the rarest elements on Earth, leading to substantial supply risks and price volatility, and are therefore designated as critical elements by the U.S. Geological Survey (USGS) [7]. This scarcity underscores the urgent need to develop efficient and cost-effective alternatives. Earth-abundant transition metals such as Ni, Fe, Ti, Mn, Cr, Zr, and V are being actively explored as substitutes [8]. Ni nanocatalysts have been widely employed in heterogeneous catalysis, and their properties can be tuned by introducing elements such as Cr, Mo, or W [9]. Pyrochlore-structured oxides, as well as mixed transition-metal oxides (e.g., NiFeOx, NiCoOx, FeCoOx), have also attracted great interest due to their diverse electrochemical behaviors and high activity toward the oxygen evolution reaction (OER) in alkaline media, with NiFeOx consistently emerging as one of the most active catalysts [10–12].

Families of multicomponent oxides incorporating PGMs or earth-abundant metals often make setting the total stoichiometry — particularly for oxygen — challenging. Such systems benefit from computational tools that enable researchers to manipulate and study materials at higher levels of abstraction. Computational materials science already provides a powerful suite of methods focused on crystal structures. Numerous reference databases exist for both theoretically computed and experimentally determined structures, including ICSD [13], OQMD [14], Materials Project [15], and Aflow [16]. Standardized engines for first-principles simulations, such as VASP [17], and molecular dynamics codes like LAMMPS [18], provide access to high-performance algorithms. Libraries such as Pymatgen [19], ASE [20], AiiDA [21], and Atomate [22] streamline atomic structure manipulation and data extraction. While Pymatgen and ASE are well-established for generating small, ordered unit cells, experimental science and precision engineering frequently demand manipulation of disordered structures across multiple length scales, capabilities that remain challenging for existing toolchains. In such settings, the ability to express a candidate structure or transformation in natural language and translate it into a computational representation without requiring direct interaction with code from crystallographic packages (e.g., "Start from fcc copper, construct a $3 \times 3 \times 3$ supercell, and substitute 30% of the Cu atoms with Ni".), would make it easier for both computational and experimental researchers to initiate their workflows.

Large Language Models (LLMs) offer a promising route toward further abstraction of such routine tasks [23]. In particular, LLMs excel at transforming unstructured natural language into structured outputs, making them promising for crystal structure generation from text prompts. Already, LLMs have demonstrated robust performance in extracting data from literature [24,25] and collecting information through data mining [26,27]. Enhanced by AI agents [25], workflows that utilize LLMs to translate natural language into computational tasks could become routine, enabling more intuitive data manipulation and increasing research productivity within human- or agent-guided frameworks.

Recent studies have also explored generative approaches for inorganic crystal structures using diffusion models, LLMs, and multimodal frameworks. MatterGen [28] employs a diffusion model trained on Materials Project and Alexandria data to generate stable structures from text prompts. LLaMP [29] combines retrieval-augmented generation (RAG) [30] with hierarchical reasoning agents [31] to edit crystal structures and run atomistic simulations. LLM-based approaches such as Crystal-LLM [32], LLMatDesign [33], AtomGPT [34], LLM-Prop [35], and Chemeleon [36] demonstrate that fine-tuned LLMs and graph neural networks can generate or analyze structures via text descriptions, properties, or property–structure mappings. Most recently, CrysText [37] fine-tuned LLaMA [38] to generate CIF files directly from space group and formula inputs.

Although these advances highlight the growing interest in text-driven materials generation, several technical barriers remain. State-of-the-art LLMs such as LLaMA, GPT, and Claude demand substantial computational resources and rely on a finite pool of high-quality human-generated text, raising concerns about data scarcity, training sustainability, and the environmental cost of large-scale model retraining. Direct CIF generation remains particularly challenging because LLMs struggle with numerical precision and physical consistency, frequently violating bond-length constraints, site occupancies, or symmetry rules. Relying on a single static model without downstream validation renders CIF outputs brittle and error-prone. Addressing these limitations requires tightly coupled post-generation correction pipelines, constraint-aware verification frameworks, and accessible interfaces that allow users to iteratively refine outputs while minimizing model-hosting risks and improving the reliability of text-driven crystal-structure generation.

Modern agentic systems — exemplified by ReAct [31], Reflexion [39], Voyager [40], AutoGPT [41], AgentVerse [42], CAMEL [43], and contemporary multi-agent frameworks from OpenAI, Anthropic, and DeepMind — commonly integrate multiple large language models within a unified reasoning and action loop [44]. Rather than relying on a single monolithic policy model, these systems combine a policy module with tool-use capabilities, an explicit plan–act–observe cycle, memory or state tracking, and an environment interface. Multi-model designs are now standard: Google's Gemini [45] agents pair a lightweight local model with a larger cloud model, mixture-of-experts and toolformer-style [46] architectures distribute tasks across specialized LLMs, and Anthropic's [47] supervisor–worker pattern operationalizes two-model collaboration. This practice demonstrates that agentic behavior emerges from coordinated components, not from a single LLM, firmly situating multi-LLM architectures within established agentic-system methodologies.

In this paper, we present an agentic framework for text-driven crystal-structure generation and manipulation, consisting of two LLM components, a code-execution environment, and a validation module, see Fig. 1. We use a small local worker LLM fine-tuned for code generation and a larger supervisor LLM for refinement. Our work addresses the listed challenges and departs from all prior text-to-crystal efforts in four essential ways. First, we generate code instead of static CIFs, enabling deterministic execution, downstream validation, and seamless integration with robotics and simulation pipelines. Second, we explicitly fine-tune the worker model to perform four core crystallographic manipulations — supercell construction, strain application, vacancy creation, and atomic substitution — triggered through natural-language commands, thereby extending beyond structure generation to structure transformation. Third, we intentionally operate with a compact CodeGen-350M-mono [48] model and apply Low-Rank Adaptation (LoRa) [49] fine-tuning, substantially reducing computational

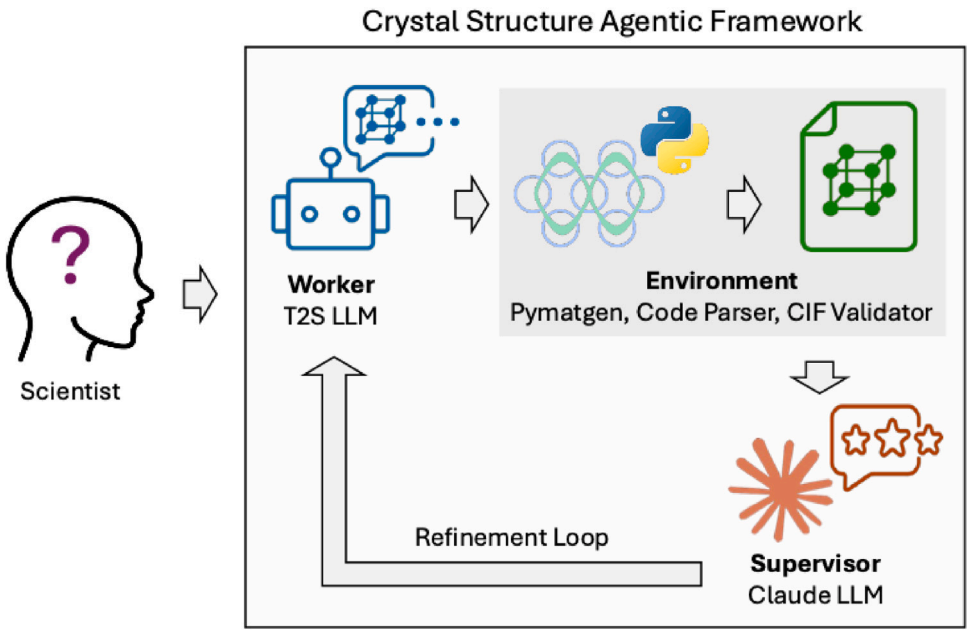


Fig. 1. Architecture of the proposed agentic framework for text-driven crystal-structure generation and manipulation. A compact fine-tuned worker LLM (CodeGen-mono-350M) generates executable Pymatgen code for structure construction and crystallographic transformations. Executed code yields serialized structure objects, which are evaluated through geometric, chemical, and symmetry validation checks. A supervisory LLM (Claude 3.5 Sonnet) evaluates intermediate results and refines user prompts, forming a self-improvement loop.

and energy demands compared to large LLMs in line with responsible AI practices. Fourth, we embed this lightweight model within an autonomous agentic framework that incorporates Claude 3.5 Sonnet for iterative self-evaluation and refinement. To support these capabilities, we curated a custom dataset of structures from Materials Project (MP) [15], expanded through systematic manipulations and multiple levels of text abstraction, and developed parsers that convert LLM outputs into serializable structure objects for quantitative evaluation. In this architecture, the small fine-tuned worker model performs long-context structure synthesis and manipulation, while the larger supervisor model is invoked sparingly for short refinement steps, providing a reliable, energy-efficient, and extensible approach to text-driven crystal-structure generation.

2. Method

Our Crystal Structure Agentic Framework integrates two complementary LLMs in a worker–supervisor configuration, a local code-execution environment, and a crystallographic validation layer (Fig. 1). A natural-language prompt describing a target structure or transformation is first handled by a compact worker model (CodeGen-mono-350M fine-tuned with LoRa), which generates executable Pymatgen code. The code is executed locally to instantiate a serializable structure object, which then undergoes geometric and chemical sanity checks. The resulting structure and reasoning trace are passed to a supervisory LLM (Claude 3.5 Sonnet), which performs high-level assessment, identifies missing or inconsistent specifications, and refines the prompt when necessary. This forms an autonomous iterative loop in which the supervisor manages planning and correction, while the compact model performs long-context structure generation and manipulation.

2.1. Worker model

To create a worker LLM, we fine-tuned the smallest model from the CodeGen family [48] using Low-Rank Adaptation (LoRa) [49] to minimize energy demands for this task. To enable fine-tuning, we curated a custom dataset designed for prompt–output text completion, where each document includes a prompt (a structure description) and an output (Pymatgen code and the corresponding CIF). CIFs were sourced from the Materials Project (MP) [15] and augmented through structural transformations (strain, supercell expansion, vacancy creation, and substitution). For each structure, we generated four types of text descriptions: (1) detailed symmetrized information, (2) a GPT-generated summary, (3) a Robocrystallographer description [50], and (4) a concise single-sentence description with formula and space group. The project for LLM fine-tuning is released on GitHub at https://github.com/ViktoriiaBaib/T2S_LLM.

2.1.1. Dataset

Each data document in the dataset consists of prompt: a text description; and output: the corresponding Pymatgen code, and/or the resulting CIF (Fig. 2). Inspired by Toolformer [46], we extend the conventional "text–CIF" mapping to "text–code–CIF", allowing the LLM to learn both code generation and structure reconstruction. To assess the effect of output format, we constructed two dataset configurations: text–code, which outputs only Pymatgen code, and text–code–CIF, which outputs both code and the resulting CIF. See SI for more information and dataset documents examples.

We retrieved 33,990 stable structures from MP (`is_stable=True`). To simplify generation, we filtered to retain only structures with both conventional and symmetrized representations, keeping those with $\leq$ 10 conventional or $\leq$ 5 symmetrized sites. We verified that rounding operations (see below) preserved conventional structures. This yielded 5581 unique compounds: 83 unary, 1912 binary, 3366 ternary, 217 quaternary, and 3 quinary (Table 1).

Dataset covers wide structural diversity, see SI. The set spans 88 out of 118 elements of the periodic table, with silicon, germanium, oxygen, nickel, palladium, copper, indium, gallium, and aluminum most frequent. All seven crystal systems and 92 space groups are represented, with I4/mmm, P63/mmc, and Pm-3 m among the most common.

To allow transformation capabilities to the model, we expanded the dataset with structures after four Pymatgen-based transformations (Fig. 3): (1) supercell, scaling factors 1–3, with $\leq$ 20 sites retained; (2) strain, linear tensile strain up to 10% per axis; (3) vacancy, single site removed

Prompt: text

Build this crystal structure with pymatgen
<filename>structure_description
LuSe crystallizes in the Fm-3m (225) space group.

Output: code

<filename>solutions/solution_1
Import pymatgen objects:

```
from pymatgen.core import Lattice, Structure
```

To build a crystal, set correct space group:

```
space_group = 'Fm-3m'
```

From it, set lattice by vectors and angles:

```
lattice = Lattice.from_parameters(a=5.22,b=5.22,c=5.22, alpha=90,beta=90,gamma=90)
```

Set atomic sites:

```
atoms = ['Lu','Se']
```

and:

```
coordinates = [[0.5, 0., 0.], [0., 0., 0.]]
```

Build a crystal:

```
crystal = Structure.from_spacegroup(space_group, lattice, atoms, coordinates)
```

Output: CIF

```
Full Formula (Lu1 Se1)
Reduced Formula: LuSe
abc   :   5.32757 5.32757 5.32757
angles:  90.000000  90.000000  90.000000
Sites (2)
  #  SP      a    b    c  Wyckoff
---  ----  ---  ---  ---  ---------
  0  Lu   0.5  0    0  1b
  1  Se    0    0    0  1a
```

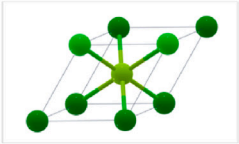


Fig. 2. Dataset document example. On the left, at the top: a prompt that includes a set prefix and text, here featuring a space group description. On the left, at the bottom: an output that includes commented Pymatgen code. On the right: the continuation of the output with a structure CIF. The LLM's learning objective is to generate an output based on the given prompt.

Table 1
Number of crystal structures in a dataset, breakdown by arity. Structures were sourced from the MP database and underwent filtration.

Arity	Total	Unary	Binary	Ternary	Quaternary	Quinary	6,7,8-ary
MP query	33,990	93	5042	19,285	7860	1527	183
Filter by n sites	5581	83	1912	3366	217	3	0

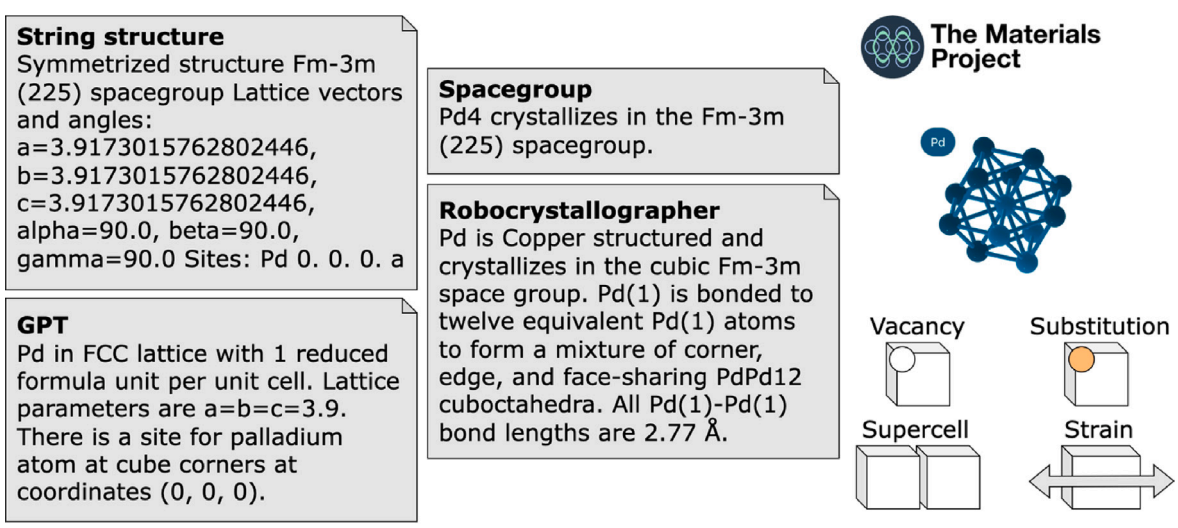


Fig. 3. Each crystal structure from the Materials Project Database yields $(1+4)\times 4 = 20$ documents: one original and four transformed structures, each paired with four text descriptions.

in conventional cell; and (4) substitution, atomic sites replaced by chosen elements at a specified percentage. All transformations produced CIFs in conventional representation.

We generated four structure descriptions differing by level of detail: (1) structure, a full symmetrized representation listing parameters for lattice vectors, angles, elements, Wyckoff sites, and fractional coordinates; (2) gpt, a summary produced with GPT-3.5-turbo (6/11/2024), temperature 0.3, chain-of-thought prompting [51], few-shot examples, and crystallography-specific instructions [52,53]; (3) robocrystallographer, a programmatic, deterministic description of motifs and bonding

capped at 400 characters [50]; and (4) spacegroup, a single-sentence with chemical formula and space group. This design balances completeness, natural variability, conciseness, and structured consistency. See SI for more details.

Pymatgen code was generated based on the usage of `from_spacegroup`. Parameters (space group, lattice, angles, elements, coordinates) were extracted from serialized structures. To reduce numerical errors and sequence length, we rounded lattice vectors to 2 decimals and angles to integers, expressed coordinates as fractions ($p/q \leq 100$ or 4-decimal floats). For each code line, we included explanatory comments; both commented and uncommented code versions were provided.

The final dataset, released on Hugging Face (https://huggingface.co/datasets/vbv373/texT2Struc_dataqa), contains 111,615 documents with a 64:16:20 train–validation–test split (by MP id, before transformations).

2.1.2. Fine-tuning

We fine-tuned CodeGen-350M-mono [48], a 357M-parameter autoregressive transformer pretrained on GitHub code and Python. The model was adapted using LoRA [49] with rank 8, α=32, and dropout 0.05, resulting in 327,680 trainable parameters (0.1%) and GPU memory footprint of 1.5 GB. Training was conducted at 16-bit precision with quantization enabled, maximum context length of 1024 tokens, using batch sizes of 24 (training) and 32 (evaluation), gradient accumulation of 4, and a learning rate of 4e-5. Logging was performed every step, evaluation every 100 steps, and training used full train/evaluation splits for 3 epochs with 100 warm-up steps. All experiments were run on a single AWS g6.2xlarge instance equipped with an NVIDIA L4 GPU (24 GB). Hyperparameters were tuned using Weights and Biases. See SI for more information.

The choice of a smaller model balances capability and resource efficiency, aligning with responsible AI practices [54,55]. Fine-tuning on our dataset of 111,615 text–code pairs for three epochs with a maximum context length of 1024 tokens corresponds to an upper-bound token exposure of 3.4e8 tokens (111,615 × 1024 ×3). For CodeGen-350M-mono with 357 M parameters, this implies an estimated fine-tuning cost of 7e17 floating-point operations (FLOPs), or 0.7 EFLOPs, using the standard Transformer compute approximation ($6 \times N$ params $\times N$ tokens) [56]. Although only 0.1% of parameters are trainable under LoRA, the forward and backward passes still traverse the full network, so this estimate captures the correct order of magnitude for compute; LoRA primarily reduces optimizer state and memory requirements and enables single-GPU fine-tuning. On an NVIDIA L4 GPU, which sustains 20–40 TFLOP/s effective throughput for mixed-precision training workloads, this corresponds to approximately 5–10 GPU-hours and 1–3 kWh of total energy, consistent with our observed training logs. Fine-tuning a larger-parameter model on the same dataset would scale approximately linearly with parameter count; for example, adapting an 8B-parameter model would require 22.4 (8/0.357) times more compute, with node-hours and energy increasing by at least this factor and typically more due to the need for multi-GPU, higher-power accelerators.

We fine-tuned on two dataset variants: text–code and text–code–CIF (Table 2). Since CIFs contain long numeric site listings, we hypothesized lower accuracy for text–code–CIF due to token noise and numerical precision issues [57]. Models are available on Hugging Face: https://huggingface.co/vbv373/test_texT2Struc_text_code_1116, https://huggingface.co/vbv373/test_texT2Struc_text_code_cif_1116

Inference was run on 5000 test-set structures (balanced across descriptions and transformations). After 10 warmup generations, outputs were collected and released(https://huggingface.co/datasets/vbv373/5000_generations_test_texT2Struc_text_code_1116, https://huggingface.co/datasets/vbv373/5000_generations_test_texT2Struc_text_code_cif_1116).

Table 2
Fine-tuned models and structure retrieval methods. Two LLMs were trained on two dataset configurations. Two different methods for structure retrieval were applied. This resulted in three generated structures for each original structure.

	Model	Retrieval	Generated Structure
(1)	text–code	API from code	api, code
(2)	text–code–CIF	API from code	api, code–CIF
(3)	text–code–CIF	parser from CIF	parser, code–CIF

We implemented two methods for retrieving structures from model generations (Table 2): (i) the API-from-code method, which parses the generated code block and executes it with Pymatgen, and (ii) the CIF parser, which reconstructs structures directly from generated CIF files. When the model generates only code, a single structure is obtained via the API-from-code method. When the model generates both code and CIF, two additional structures are produced: one via the API-from-code method and another via the CIF parser. Both methods were evaluated on the same test set sample with two models. Consequently, for each original structure three generated structures are obtained: (1) API-from-code using text–code, (2) API-from-code using text–code–CIF, and (3) CIF parser using text–code–CIF. We note that the expected order of accuracy is: API-from-code (text–code) > API-from-code (text–code–CIF) > CIF parser (text–code–CIF).

2.1.3. Evaluation

Generated structures were compared to references using Pymatgen's Structure Matcher with the Order–Disorder Element Comparator [28], with matches defined by an RMS distance < 0.1. Non-matching but parsable structures were retained as valid generations, while unparsable outputs were classified as hallucinations. In this work, hallucination refers strictly to syntactic or execution-level failure (e.g., invalid API calls, incompatible space group–lattice combinations, or CIF parsing errors). Chemically implausible but parsable structures were not categorized as hallucinations; instead, they were treated as valid but non-matching generations under StructureMatcher evaluation. Additional quantitative metrics capturing partial structural and compositional agreement and full parameter details are provided in the SI (Figure S4)†. Evaluation datasets are available on Hugging Face.

2.2. Supervisor model and refinement loop

The supervisory model (Claude 3.5 Sonnet) functions as the high-level controller that evaluates worker-generated structures, determines whether the output satisfies the user's intent, and autonomously refines the prompt when discrepancies arise. After the worker LLM produces Pymatgen code and the code is executed to construct a structure object, we extract key descriptors — composition, lattice type (e.g., fcc, bcc, hcp), and basic geometric parameters — using Pymatgen's built-in analysis functions. These descriptors, together with the original user prompt, are provided to the supervisory LLM in a structured message. Through targeted prompt engineering, the supervisor is instructed to (i) identify which specifications are explicitly present in the original prompt, (ii) compare these specifications against the generated structure's composition and lattice characteristics, and (iii) reason about whether the worker's output aligns with the intended target. The supervisor is further informed of the worker model's supported crystallographic transformations (supercell scaling, strain, vacancy creation, substitution), enabling it to determine which manipulations are available for correcting deviations. Based on this comparison, the supervisory model autonomously decides whether the prompt should be modified, how it should be modified, and whether the worker should be called again. This establishes a closed-loop refinement cycle in which the supervisor performs planning, consistency checks, and prompt adjustment, while the worker generates updated structures until convergence or termination criteria are met.

In the current implementation, the refinement loop is intentionally shallow. Rather than enabling indefinite autonomous back-and-forth interaction, the supervisory model performs a single diagnostic pass in which it compares generated structural descriptors to the user's intent and proposes targeted corrective transformations supported by the worker (e.g., substitution, strain, supercell). The worker is then recalled once with the refined instruction. This design reflects a controlled proof-of-concept agentic pattern, emphasizing deterministic tool use and minimal additional inference overhead rather than open-ended multi-turn reasoning. While the architecture can in principle support deeper iterative loops, we intentionally restrict it to a single supervisory correction step to preserve transparency, reproducibility, and computational efficiency.

3. Results

Quantitative evaluation of model performance is reported in Fig. 4, where we benchmark single-pass generations on 5000 held-out test structures using hallucination rate and StructureMatcher exact-match accuracy across generation modes, prompt types, and transformations (e.g., 5%–14% execution failures, 36% match rate for the best overall generation mode, and up to 82% match accuracy for fully specified 'structure' prompts). We generated outputs from each model in a single pass, without sampling for optimal generations. Fine-tuned LLMs successfully produced code or CIF files that converted to valid crystal structures in most cases (see Fig. 4), with hallucination rates — parser failures due to errors such as "supplied space group is incompatible with the supplied lattice parameters" — ranging from 5%–14%. By contrast, the untrained CodeGen-350M-mono model hallucinated 100% of the time under identical prompts, consistently producing meaningless outputs (e.g., True/False or single numbers) or invalid pseudocalls, even with partial outputs or guidance (see S.I.). This underscores the impact of fine-tuning on task performance.

We evaluated three generation modes against the original structures, see Fig. 4A: (1) API, code — executing code from the text–code LLM; (2) API, code–CIF — executing code from the text–code–CIF LLM; and (3) CIF parser, code–CIF — parsing the CIF from the text–code–CIF LLM. Pymatgen's StructureMatcher determined match rates. API execution yielded the highest match rates: 36% for "API, code" and 30% for "API, code+CIF", versus 8% for "CIF parser, code+CIF". Hallucination rates were lowest for "API, code" (5%) and highest for "API, code+CIF" (14%), while CIF parsing reduced failures to 4% by avoiding space group constraints. Overall, "API, code" was the best-performing method.

Compared to direct CIF generation, the code-based outputs reduced average sequence length by 5–10 times and enabled four distinct structure transformations to be executed programmatically, demonstrating that the model had learned actionable crystallographic logic beyond text–CIF mapping.

We then focused on best performing generation mode API, code and assessed the performance across four prompt types, see Fig. 4B: (1) structure — full lattice parameters and sites; (2) GPT — partial parameters requiring extrapolation; (3) Robocrystal — formula, space group, and extraneous descriptors; (4) Spacegroup — only formula and space group. With all sites provided ("structure"), match accuracy reached 82% with only 2% execution failures. Performance declined with reduced parameter detail: 44% matches and 11% failures for "GPT", and 8%–9% matches with 3%–4% failures for "Robocrystal" and "Spacegroup". Extra descriptive but non-codable details in "Robocrystal" did not improve performance. When sites were missing, the LLM occasionally inferred them, but success likely depends on structural similarity to training examples.Continuous metrics (volume and composition; see SI†) confirmed the same trend: "API, code" produced the smallest deviations from reference structures. CIF printing likely underperformed because the baseline model was originally trained for code generation. Prior work with larger models [58] achieved strong structure generation, but our focus on structure transformations, see Fig. 4C, made the task more challenging—substitution transformations, for instance, had roughly half the success rate of others.

3.1. Use case

The following catalyst case studies are presented to illustrate practical tool behavior and iterative refinement pathways (generation - validation - optional transformation/relaxation), rather than to provide additional quantitative scoring beyond the global metrics above. Two modes of use are envisioned: (i) specifying detailed crystallographic parameters, and (ii) relying on the model to infer plausible structures. We demonstrate that for complex structures, our agentic framework can generate an initial approximation close to the desired target.

Our worker LLM, also referred as T2S, possesses crystallographic knowledge for relatively simple structures and can perform operations such as supercell scaling, applying strain, creating vacancies, and substituting atoms up to a specified percentage. As a use case, we aim to generate structures that match requested stoichiometries. The agentic workflow combines worker LLM for generation with supervisor LLM for evaluating the execution results and suggesting further steps toward the desired configuration. In all cases, the generated structures are treated as initial approximations requiring subsequent geometry relaxation.

We first consider IrO_2, a well-known noble metal benchmark catalyst for OER, benefiting from the PGM properties of Ir [2,5,6]. Although IrO_2 exhibits variations such as IrO(OH) and IrO_x, its configuration is relatively straightforward. By providing detailed crystallographic parameters from the Materials Project database as input, the worker model successfully generates an accurate structure, see Fig. 5A.

The second example is $Pb_2Ir_2O_7$, a multimetal oxide containing the PGM element Ir in a pyrochlore structure [1,11]. This compound is present in the Materials Project database (mp-1190246) and features significant complexity: four symmetrized sites with site multiplicities of 16 for metals and 48 for one of the oxygen atoms. Instead, we prompt the worker model with the formula $Pb_2Ir_2O_7$ and the description of a pyrochlore structure: "Pb occupies the 16d position (1/2, 1/2, 1/2), Ir occupies the 16c site at (0,0,0), and oxygen atoms are distributed between the 48f $(x, 1/8, 1/8)$ and 8b $(3/8, 3/8, 3/8)$ Wyckoff positions. Oxygen at the 48f site [site symmetry mm (C_{2v})] forms a distorted tetrahedron with two Pb and two Ir neighbors". The generated structure (Fig. 5B) is considerably simpler and smaller than the target, which is expected because the model was trained primarily on structures with up to five symmetrized sites. As structure complexity increases, the LLM exhibits more frequent hallucinations. Nevertheless, the supervisor model identifies that the correct stoichiometry can be achieved by iteratively invoking T2S with supercell transformations and vacancy creation.

The third example involves $NiFeO_x$, an effective multimetal oxide catalyst composed of earth-abundant transition metals [8,10,12]. Nickel oxides are promising OER catalysts, and their performance can be enhanced by incorporating Fe. These compounds exhibit variable stoichiometry, such as $NiFeO_x$ and $NiFeO(OH)_x$, with preparation ratios of Ni:Fe ranging from 2:1 to 16:1 or beyond. Further improvements can result from introducing trace amounts of rare earth elements such as Re. For this study, we select Ni_2FeO_4, providing the model with group and coordination details: "$FeNi_2O_4$ crystallizes in the trigonal space group $R-3m$, where Fe forms FeO_4 tetrahedra sharing corners with NiO_6 octahedra". The resulting structure is valid and exhibits the correct atomic ratio, although the atomic arrangement is layered rather than strictly tetrahedral, see Fig. 5C.

The fourth example is the NiMo catalyst [9]. We choose a Ni : Mo ratio of 3:1 and prompt the model with: "Ni_3Mo crystallizing in the Pmmn space group". The generated structure contains Ni_2Mo_2 and requires substituting one Mo atom with Ni (Fig. 5D). Furthermore, the cell vectors are initially equal and must be strained to satisfy the orthorhombic symmetry. These corrections can be accomplished by iterating worker model with appropriate adjustments.

Across all cases, the worker model demonstrates a robust ability to generate valid crystal structures, with lattice parameters internally consistent. When stoichiometry is incorrect, the supervisor model suggests how the desired configuration can be obtained by iterative prompting combined with the worker model's built-in structure manipulation capabilities.

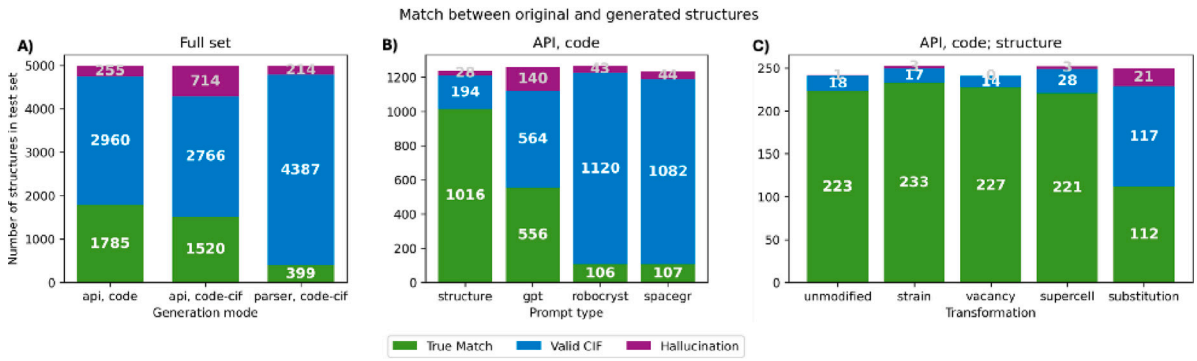


Fig. 4. Fine-tuned LLM performance as number of exactly matched structures: (a) across model generation and structure parsing modes; (b) for the best performing mode, across prompt types; (c) for the best performing mode and prompt type, across transformations.

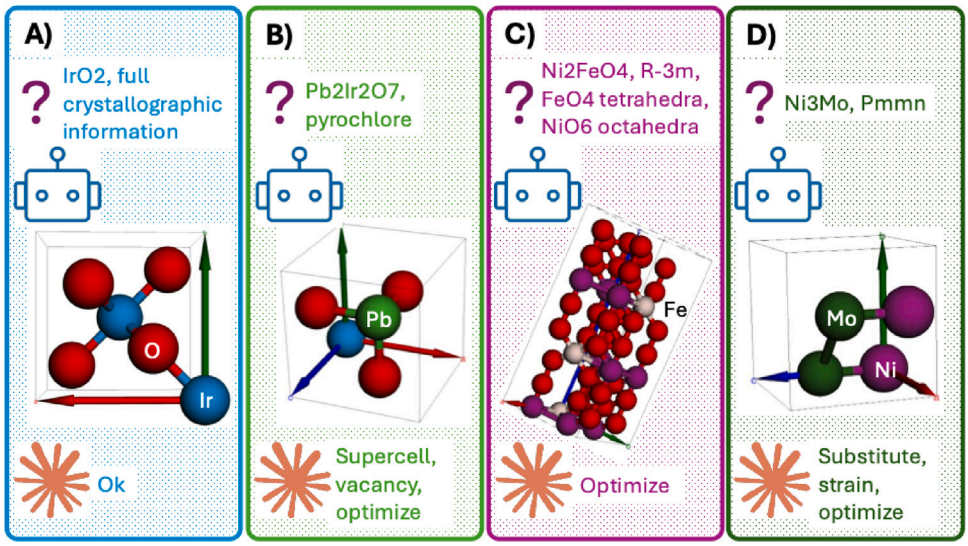


Fig. 5. AI Crystal Structure Agentic Framework use case: match requested stoichiometry for catalysts (a) IrO_2; (b) $Pb_2Ir_2O_7$; (c) Ni_2FeO_4; (d) Ni_3Mo.

4. Discussion

Our results demonstrate that fine-tuned LLMs can generate crystal structures through API calls with partial adherence to physical constraints. Formatting structure generation as programmatic instructions enables LLMs to produce computationally usable crystal structures, suggesting utility for automated materials workflows.

We developed an agentic, tool-oriented LLM framework that generates crystal structures by producing executable Pymatgen code — rather than emitting CIFs directly — yielding compact, verifiable representations that align with laboratory automation workflows. This design is fundamentally superior to direct CIF generation from large LLMs because code execution enforces crystallographic and chemical constraints deterministically, eliminating the numeric drift, broken symmetry, invalid occupancies, and unphysical bond lengths that large models frequently introduce when generating CIF text token-by-token. Code also provides transparent intermediate states, structured error handling, and seamless integration with simulation or robotic pipelines—capabilities that static CIF strings cannot offer.

Unlike direct CIF generation, our code-based strategy embeds the procedural logic of crystallographic construction directly into the model's learning process. By training the LLM to emit executable Pymatgen commands rather than static coordinate files, we enable it to internalize how crystal structures are built and modified—not merely represented. This abstraction grants the model the ability to apply operations such as supercell scaling, strain, vacancy creation, and atomic substitution in response to natural-language prompts, effectively merging code synthesis with structural reasoning. These manipulations, which are difficult or impossible to express in a single CIF file, are executed programmatically and can be chained into iterative design workflows. Consequently, the LLM learns both representation and action space over materials, serving as a bridge between text reasoning and atomistic simulation. This design also promotes transparency and interoperability: generated scripts are compact (5–10 times shorter in token count than CIFs), human-verifiable, and directly executable in standard materials toolkits. Instead of opaque coordinate lists, researchers can inspect and modify high-level commands such as from spacegroup or apply strain, enabling rapid debugging, composability, and integration into computational pipelines. In contrast, direct CIF generation produces static, non-interactive outputs that obscure generation logic and preclude agentic refinement. Therefore, code generation represents not a minor representational adjustment, but a functional transition toward interpretable, extensible, and manipulable AI-assisted materials design.

This approach offers a quick way to construct crystal structures. Our approach reduces reliance on direct LLM generation of large structural files. Instead, the LLM produces Pymatgen code [19], which represents crystal structure compactly, facilitates structure manipulation, and simplifies human verification. For instance, it is easier to check a call

generating a supercell than to manually inspect a CIF with hundreds of atoms. This ensures efficiency while keeping human oversight feasible.

We emphasize that the primary technical contribution of this work lies in the preparation of the crystallographic text–code dataset and the fine-tuning of the compact worker model, which required extensive data curation, transformation design, validation tooling, and multi-stage training over an extended development period. The supervisory component is intentionally lightweight and serves as an illustrative use-case of how the fine-tuned worker can be embedded within a broader agentic workflow. It demonstrates compatibility with existing supervisory LLM infrastructure.

A direct comparison with Claude Haiku 4.5 acting as a standalone generator (see S.I. for examples) revealed that general-purpose LLMs tend to produce verbose outputs, pseudocode-style lattice constructions inconsistent with Pymatgen's API, heuristic atomic coordinates without symmetry enforcement, incomplete stoichiometry under multiplicity constraints, and frequent truncation at fixed token limits. Producing reliable crystallographic code required extensive prompt conditioning specifying symmetry and formatting rules. In contrast, the fine-tuned worker generates compact, deterministic 'from spacegroup' constructions aligned with the training distribution, highlighting the benefit of domain specialization. Crystallographic structure generation is a constrained symbolic task requiring deterministic API alignment and symmetry enforcement. Domain adaptation reduces variability and improves structural validity.

Our approach compares to recent advances in text-driven crystal structure generation, most notably CrysText [37], which fine-tuned an 8B-parameter LLaMA model [38] to produce CIF-formatted structures from space groups and formulas. In contrast, our fine-tuned 350M-parameter model — originally trained for code generation — achieves crystallographic reasoning and structural manipulation with a far smaller computational footprint. This demonstrates that domain adaptation of compact models can yield meaningful functionality without relying on massive foundation models, aligning with responsible AI principles of minimizing energy demands while maintaining scientific utility.

Beyond construction, LLMs could extract structures from literature, fill in incomplete structural data, and link published descriptions to databases such as Materials Project or ICSD [24,59,60]. When combined with property-extraction tools such as LLM-Prop [35], this may help overcome data limitations in structure–property relationships. Moreover, text-driven generation could assist in hypothesis formation by producing plausible candidate structures for poorly characterized compounds [61,62].

Several limitations remain. First, T2S supports only a narrow set of transformations (supercells, strain, vacancies, doping). Expanding capabilities requires further data curation and fine-tuning. Second, the dataset contains simplified structures, which limit the model's ability to generalize to complex crystallography. Unseen space groups or symmetry operations are particularly error-prone unless explicitly specified by the user. Third, the range of text descriptions is limited; novel phrasing may reduce performance.

A key limitation of the present T2S worker LLM is that its training distribution was intentionally biased toward small, ordered unit cells: to simplify sequence length and stabilize code generation, we filtered Materials Project structures to those with both conventional and symmetrized representations and ≤ 10 conventional or ≤ 5 symmetrized sites, yielding 5581 unique compounds for training. This design choice improves reliability for compact prototypes but imposes a strong prior that "valid crystals" resemble low-site-count motifs, which materially constrains applicability to complex functional materials with high Wyckoff multiplicities, large asymmetric units, or subtle symmetry/parameter couplings. The use cases explicitly expose this boundary: for $Pb_2Ir_2O_7$ (pyrochlore), the worker generates a "considerably simpler and smaller" structure, which is expected under a $\leq$ 5-symmetrized-site training regime and becomes more error-prone as structural complexity increases. Likewise, for Ni_3Mo (Pmmn) the initial output has incorrect stoichiometry (Ni_2Mo_2) and an over-symmetrized metric (equal cell vectors), requiring downstream corrective operations (substitution + strain) to reach the intended orthorhombic target. We therefore emphasize that the current worker LLM should be interpreted primarily as a structure seeding and manipulation tool for relatively simple ordered cells — useful for rapid initialization and subsequent relaxation — rather than a stand-alone generator for novel, crystallographically complex compounds; expanding coverage will require retraining on higher-site-count structures (including complex oxides and larger asymmetric units) and/or incorporating explicit symmetry- and multiplicity-aware constraints in the generation/validation loop.

To reduce errors, several strategies can be considered. Hybrid training, augment text-based fine-tuning with crystallography-constrained loss using executed code comparisons. Iterative refinement, couple LLM outputs with energy minimization at added computational cost. Multimodal integration, combine text with partial structural inputs. Explainability layers, provide reasoning for structural generations, albeit with added complexity. Explicit chemical priors, encode atomic radii, electronegativity, or similar knowledge directly into the model. Adaptive sampling, prioritize uncertain regions of generation space. These trade-offs balance accuracy, efficiency, and novelty in structure prediction.

In an era of increasingly large AI models, our approach deliberately relied on a relatively small LLM. This choice reflects a commitment to responsible AI by minimizing training and inference energy demands. Despite its modest size, the fine-tuned model successfully learned core crystallographic principles and structure manipulation tasks, even if its performance was limited. These results demonstrate that enormous foundation models are not always necessary for domain-specific scientific applications: carefully fine-tuned smaller models incorporated into a workflow can provide meaningful functionality with a far lower computational footprint.

On a broader perspective, LLMs exhibit three central limitations: hallucinations [63], sensitivity to input formulation, and constraints of model size/domain knowledge. Hallucinations reflect the model's inability to discriminate between plausible and implausible patterns. Risks can be mitigated by careful dataset design, scope limitation, and human-verifiable outputs. Input sensitivity underscores the importance of prompt engineering [64,65], as model performance degrades when prompts diverge from training distributions. Finally, model selection is nontrivial: smaller models offer efficiency but reduced generalizability, while large models are expensive to train and adapt [66]. Fine-tuning methods such as LoRa [49] offer practical middle ground, as demonstrated by related efforts [37].

5. Conclusion

We present an AI-driven approach for generating crystal structures from natural language by fine-tuning a 350M-parameter LLM to output Pymatgen code, enabling efficient and interpretable structure construction. Fine-tuning reduced hallucination rates from 100% to 5% and achieved up to 82% structural match accuracy for fully specified prompts, with reasonable performance even under sparse input. The model supports key manipulations — supercell scaling, strain, substitution, and vacancies — facilitating iterative refinement and automated workflows. Demonstrations on catalysts such as IrO_2, $Pb_2Ir_2O_7$, Ni_2FeO_4, and Ni_3Mo highlight applicability to PGM-based and earth-abundant systems. Our results show that small, domain-adapted models can deliver practical crystallographic reasoning with a fraction of the energy footprint of billion-scale LLMs.

To enhance reliability and generalization, future work should incorporate chemically diverse datasets, crystallographic constraints, and hybrid training with energy-minimized feedback. Combining text prompts with partial structural inputs or multimodal priors can reduce uncertainty, while iterative loops integrating property evaluation can

guide optimization toward functional targets. Embedding explicit chemical knowledge and adaptive sampling strategies will further balance efficiency and accuracy. Ultimately, integrating lightweight LLMs into agentic frameworks with simulation backends could enable closed-loop design pipelines, accelerating materials discovery under sustainable AI principles.

CRediT authorship contribution statement

Viktoriia Baibakova: Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Alexey Serov:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work the authors used ChatGPT in order to avoid grammatical errors. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Viktoriia Baibakova reports financial support and writing assistance were provided by Toyota Research Institute. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

V.B. and A.S. acknowledge a financial support from US DOE Roll-to-Roll Consortium.

V.B. also thanks the Toyota Research Institute for financial support during a 12-week Summer 2023 internship, and access to resources. V.B. is grateful for the mentoring, technical contributions, and discussions from Steven B. Torrisi, Weike Ye, Igor Vasiljevic, Sedrick Keh, Joseph Montoya, and Linda Hung (Toyota Research Institute), and for valuable project vision suggestions from Anubhav Jain (Lawrence Berkeley National Laboratory) and Matthew Horton (Microsoft).

This manuscript has been authored in part by UT-Battelle, LLC, under Contract No. DE-AC05-00OR22725 with the U.S. Department of Energy (DOE). The U.S. Government retains and the publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this manuscript, or allow others to do so, for U.S. Government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (http://energy.gov/downloads/doe-public-access-plan).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.egyai.2026.100710.

Data availability

Data and code will be publicly available. Links included in the paper.

References

[1] Su Lixin, Wu Hao, Zhang Shaokun, Cui Chenxi, Zhou Shengnan, Pang Huan. Insight into intermediate behaviors and design strategies of platinum group metal-based alkaline hydrogen oxidation catalysts. Adv Mater 2025;37(4):2414628.
[2] Du Wei, Zhang Wei, Zhu Chengxin, Guo Wenjin, He Meiqi, Zhao Huiping, Chen Rong. Platinum group metals-based intermetallic compounds: Syntheses and application in electrocatalysis. Coord Chem Rev 2025;530:216473.
[3] Phogat Peeyush, Chand Bhawana, Jha Ranjana, Singh Sukhvir, et al. Hydrogen and methanol fuel cells: A comprehensive analysis of challenges, advances, and future prospects in clean energy. Int J Hydrog Energy 2025;109:465–85.
[4] Qi Junlei. Metal oxides and related solids for electrocatalytic water splitting. Elsevier; 2022.
[5] Wei Chao, Wang Zhenbin, Otani Kanan, Hochfilzer Degenhart, Zhang Ke, Nielsen Rasmus, Chorkendorff Ib, Kibsgaard Jakob. Benchmarking electrocatalyst stability for acidic oxygen evolution reaction: the crucial role of dissolved ion concentration. ACS Catal 2023;13(21):14058–69.
[6] Baik Chaekyung, Lee Seung Woo, Pak Chanho. Glycine-induced ultrahigh-surface-area iro2@ IrOx catalyst with balanced activity and stability for efficient water splitting. Electrochim Acta 2021;390:138885.
[7] Lanka Sri. 2020–2021 minerals yearbook. US Geol Surv 2025.
[8] Wang Dong, Astruc Didier. The recent development of efficient earth-abundant transition-metal nanocatalysts. Chem Soc Rev 2017;46(3):816–54.
[9] Yang Kangkang, Yao Qilu, Huang Wei, Chen Xiangshu, Lu Zhang-Hui. Enhanced catalytic activity of NiM (M=Cr, Mo, W) nanoparticles for hydrogen evolution from ammonia borane and hydrazine borane. Int J Hydrog Energy 2017;42(10):6840–50.
[10] Song Fang, Busch Michael M, Lassalle-Kaiser Benedikt, Hsu Chia-Shuo, Petkucheva Elitsa, Bensimon Michaël, Chen Hao Ming, Corminboeuf Clemence, Hu Xile. An unconventional iron nickel catalyst for the oxygen evolution reaction. ACS Central Sci 2019;5(3):558–68.
[11] Fuentes Antonio F, O'Quinn Eric C, Montemayor Sagrario M, Zhou Haidong, Lang Maik, Ewing Rodney C. Pyrochlore-type lanthanide titanates and zirconates: Synthesis, structural peculiarities, and properties. Appl Phys Rev 2024;11(2).
[12] Osmieri Luigi, Yu Haoran, Hermann Raphaël P, Kreider Melissa E, Meyer III Harry M, Kropf A Jeremy, Park Jae Hyung, et al. Aerogel-derived nickel-iron oxide catalysts for oxygen evolution reaction in alkaline media. Appl Catal B: Environ Energy 2024;348:123843.
[13] Zagorac Dejan, Müller H, Ruehl S, Zagorac J, Rehme Silke. Recent developments in the inorganic crystal structure database: theoretical crystal structure data and related features. J Appl Crystallogr 2019;52(5):918–25.
[14] Saal James E, Kirklin Scott, Aykol Muratahan, Meredig Bryce, Wolverton Christopher. Materials design and discovery with high-throughput density functional theory: the open quantum materials database (OQMD). Jom 2013;65:1501–9.
[15] Jain Anubhav, Ong Shyue Ping, Hautier Geoffroy, Chen Wei, Richards William Davidson, Dacek Stephen, Cholia Shreyas, et al. Commentary: The materials project: A materials genome approach to accelerating materials innovation. APL Mater 2013;1(1).
[16] Curtarolo Stefano, Setyawan Wahyu, Hart Gus LW, Jahnatek Michal, Chepulskii Roman V, Taylor Richard H, Wang Shidong, et al. AFLOW: An automatic framework for high-throughput materials discovery. Comput Mater Sci 2012;58:218–26.
[17] Hafner Jürgen. Ab-initio simulations of materials using VASP: Density-functional theory and beyond. J Comput Chem 2008;29(13):2044–78.
[18] Thompson AP, Aktulga HM, Berger R, Bolintineanu DS, Brown WM, Crozier PS, in 't Veld PJ, Kohlmeyer A, Moore SG, Nguyen TD, Shan R, Stevens MJ, Tranchida J, Trott C, Plimpton SJ. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. Comp. Phys Comm 2022;271:108171. http://dx.doi.org/10.1016/j.cpc.2021.108171.
[19] Ong Shyue Ping, Richards William Davidson, Jain Anubhav, Hautier Geoffroy, Kocher Michael, Cholia Shreyas, Gunter Dan, Chevrier Vincent L, Persson Kristin A, Ceder Gerbrand. Python materials genomics (pymatgen): A robust, open-source python library for materials analysis. Comput Mater Sci 2013;68:314–9.
[20] Larsen Ask Hjorth, Mortensen Jens Jørgen, Blomqvist Jakob, Castelli Ivano E, Christensen Rune, Dułak Marcin, Friis Jesper, et al. The atomic simulation environment—a Python library for working with atoms. J Phys: Condens Matter 2017;29(27):273002.
[21] Uhrin Martin, Huber Sebastiaan P, Yu Jusong, Marzari Nicola, Pizzi Giovanni. Workflows in AiiDA: Engineering a high-throughput, event-based engine for robust and modular computational workflows. Comput Mater Sci 2021;187:110086.
[22] Mathew Kiran, Montoya Joseph H, Faghaninia Alireza, Dwarakanath Shyam, Aykol Muratahan, Tang Hanmei, Chu Iek-heng, et al. Atomate: A high-level interface to generate, execute, and analyze computational materials science workflows. Comput Mater Sci 2017;139:140–52.
[23] Zimmermann Yoel, Bazgir Adib, Afzal Zartashia, Agbere Fariha, Ai Qianxiang, Alampara Nawaf, Al-Feghali Alexander, et al. Reflections from the 2024 large language model (LLM) hackathon for applications in materials science and chemistry. 2024, arXiv preprint arXiv:2411.15221.

[24] Dagdelen John, Dunn Alexander, Lee Sanghoon, Walker Nicholas, Rosen Andrew S, Ceder Gerbrand, Persson Kristin A, Jain Anubhav. Structured information extraction from scientific text with large language models. Nat Commun 2024;15(1):1418.

[25] Ansari Mehrad, Moosavi Seyed Mohamad. Agent-based learning of materials datasets from scientific literature. 2023, arXiv preprint arXiv:2312.11690.

[26] Zheng Zhiling, Zhang Oufan, Borgs Christian, Chayes Jennifer T, Yaghi Omar M. ChatGPT chemistry assistant for text mining and the prediction of MOF synthesis. J Am Chem Soc 2023;145(32):18048–62.

[27] Suvarna Manu, Vaucher Alain Claude, Mitchell Sharon, Laino Teodoro, Pérez-Ramírez Javier. Language models and protocol standardization guidelines for accelerating synthesis planning in heterogeneous catalysis. Nat Commun 2023;14(1):7964.

[28] Zeni Claudio, Pinsler Robert, Zügner Daniel, Fowler Andrew, Horton Matthew, Fu Xiang, Wang Zilong, et al. A generative model for inorganic materials design. Nature 2025;1–3.

[29] Chiang Yuan, Chou Chia-Hong, Riebesell Janosh. LLaMP: Large language model made powerful for high-fidelity materials knowledge retrieval and distillation. 2024, arXiv preprint arXiv:2401.17244.

[30] Lewis Patrick, Perez Ethan, Piktus Aleksandra, Petroni Fabio, Karpukhin Vladimir, Goyal Naman, Küttler Heinrich, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Adv Neural Inf Process Syst 2020;33:9459–74.

[31] Yao Shunyu, Zhao Jeffrey, Yu Dian, Du Nan, Shafran Izhak, Narasimhan Karthik R, Cao Yuan. React: Synergizing reasoning and acting in language models. In: The eleventh international conference on learning representations. 2022.

[32] Gruver Nate, Sriram Anuroop, Madotto Andrea, Wilson Andrew Gordon, Zitnick C Lawrence, Ulissi Zachary. Fine-tuned language models generate stable inorganic materials as text. 2024, arXiv preprint arXiv:2402.04379.

[33] Jia Shuyi, Zhang Chao, Fung Victor. LLMatDesign: Autonomous materials discovery with large language models. 2024, arXiv preprint arXiv:2406.13163.

[34] Choudhary Kamal. AtomGPT: Atomistic generative pretrained transformer for forward and inverse materials design. J Phys Chem Lett 2024;15(27):6909–17. http://dx.doi.org/10.1021/acs.jpclett.4c01126.

[35] Rubungo Andre Niyongabo, Arnold Craig, Rand Barry P, Dieng Adji Bousso. Llm-prop: Predicting physical and electronic properties of crystalline solids from their text descriptions. 2023, arXiv preprint arXiv:2310.14029.

[36] Park Hyunsoo, Onwuli Anthony, Walsh Aron. Exploration of crystal chemical space using text-guided generative artificial intelligence. 2024, http://dx.doi.org/10.26434/chemrxiv-2024-rw8p5, ChemArxiv preprint.

[37] Mohanty Trupti, Mehta Maitrey, Sayeed Hasan M, Srikumar Vivek, Sparks Taylor D. CrysText: A generative AI approach for text-conditioned crystal structure generation using LLM. 2024, http://dx.doi.org/10.26434/Chemrxiv-2024-Gjhpq, ChemArxiv preprint.

[38] Touvron Hugo, Martin Louis, Stone Kevin, Albert Peter, Almahairi Amjad, Babaei Yasmine, Bashlykov Nikolay, et al. Llama 2: Open foundation and fine-tuned chat models. 2023, arXiv preprint arXiv:2307.09288.

[39] Shinn Noah, Cassano Federico, Gopinath Ashwin, Narasimhan Karthik, Yao Shunyu. Reflexion: Language agents with verbal reinforcement learning. Adv Neural Inf Process Syst 2023;36:8634–52.

[40] Wang Guanzhi, Xie Yuqi, Jiang Yunfan, Mandlekar Ajay, Xiao Chaowei, Zhu Yuke, Fan Linxi, Anandkumar Anima. Voyager: An open-ended embodied agent with large language models. 2023, arXiv preprint arXiv:2305.16291.

[41] Significant Gravitas. Auto-GPT: An autonomous GPT-4 experiment. 2023, URL https://github.com/Significant-Gravitas/Auto-GPT.

[42] Chen Weize, Su Yusheng, Zuo Jingwei, Yang Cheng, Yuan Chenfei, Chan Chi-Min, Yu Heyang, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In: The twelfth international conference on learning representations. 2023.

[43] Li Guohao, Hammoud Hasan, Itani Hani, Khizbullin Dmitrii, Ghanem Bernard. Camel: Communicative agents for" mind" exploration of large language model society. Adv Neural Inf Process Syst 2023;36:51991–2008.

[44] Yao Lance, Samantray Suman, Ghosh Ayana, Roccapriore Kevin, Kovarik Libor, Allec Sarah, Ziatdinov Maxim. Operationalizing serendipity: Multi-agent AI workflows for enhanced materials characterization with theory-in-the-loop. 2025, arXiv preprint arXiv:2508.06569.

[45] Team Gemini, Anil Rohan, Borgeaud Sebastian, Alayrac Jean-Baptiste, Yu Jiahui, Soricut Radu, Schalkwyk Johan, et al. Gemini: a family of highly capable multimodal models. 2023, arXiv preprint arXiv:2312.11805.

[46] Schick Timo, Dwivedi-Yu Jane, Dessì Roberto, Raileanu Roberta, Lomeli Maria, Hambro Eric, Zettlemoyer Luke, Cancedda Nicola, Scialom Thomas. Toolformer: Language models can teach themselves to use tools. Adv Neural Inf Process Syst 2024;36.

[47] Anthropic PBC. Introducing claude 3.5 sonnet. 2024, URL https://www.anthropic.com/news/claude-3-5-sonnet.

[48] Nijkamp Erik, Pang Bo, Hayashi Hiroaki, Tu Lifu, Wang Huan, Zhou Yingbo, Savarese Silvio, Xiong Caiming. Codegen: An open large language model for code with multi-turn program synthesis. 2022, arXiv preprint arXiv:2203.13474.

[49] Hu Edward J, Shen Yelong, Wallis Phillip, Allen-Zhu Zeyuan, Li Yuanzhi, Wang Shean, Wang Lu, Chen Weizhu. Lora: Low-rank adaptation of large language models. 2021, arXiv preprint arXiv:2106.09685.

[50] Ganose Alex M, Jain Anubhav. Robocrystallographer: automated crystal structure text descriptions and analysis. MRS Commun 2019;9(3):874–81.

[51] Wei Jason, Wang Xuezhi, Schuurmans Dale, Bosma Maarten, Xia Fei, Chi Ed, Le Quoc V, Zhou Denny, et al. Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 2022;35:24824–37.

[52] Bsharat Sondos Mahmoud, Myrzakhan Aidar, Shen Zhiqiang. Principled instructions are all you need for questioning llama-1/2, gpt-3.5/4. 2023, arXiv preprint arXiv:2312.16171.

[53] Bodonhelyi Anna, Bozkir Efe, Yang Shuo, Kasneci Enkelejda, Kasneci Gjergji. User intent recognition and satisfaction with large language models: A user study with chatgpt. 2024, arXiv preprint arXiv:2402.02136.

[54] Liu Zechun, Zhao Changsheng, Iandola Forrest, Lai Chen, Tian Yuandong, Fedorov Igor, Xiong Yunyang, et al. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. 2024, arXiv preprint arXiv:2402.14905.

[55] Xu Jiajun, Li Zhiyuan, Chen Wei, Wang Qun, Gao Xin, Cai Qi, Ling Ziyuan. On-device language models: A comprehensive review. 2024, arXiv preprint arXiv:2409.00088.

[56] Anthony Quentin, Biderman Stella, Schoelkopf Hailey. Transformer math 101. 2023, URL https://blog.eleuther.ai/transformer-math/.

[57] Zhu Fangwei, Dai Damai, Sui Zhifang. Language models know the value of numbers. 2024, ArXiv Preprint.

[58] Flam-Shepherd Daniel, Aspuru-Guzik Alán. Language models can generate molecules, materials, and protein binding sites directly in three dimensions as xyz, cif, and pdb files. 2023, arXiv preprint arXiv:2305.05708.

[59] Weston Leigh, Tshitoyan Vahe, Dagdelen John, Kononova Olga, Trewartha Amalie, Persson Kristin A, Ceder Gerbrand, Jain Anubhav. Named entity recognition and normalization applied to large-scale information extraction from the materials science literature. J Chem Inf Model 2019;59(9):3692–702.

[60] Wu Siyuan, Zhu Tiannian, Tu Sijia, Xiao Ruijuan, Yuan Jie, Wu Quansheng, Li Hong, Weng Hongming. Literature classification and its applications in condensed matter physics and materials science by natural language processing. Chin Phys B 2024;33(5):050704.

[61] Sayeed Hasan M, Baird Sterling G, Sparks Taylor D. Structure feature vectors derived from robocrystallographer text descriptions of crystal structures using word embeddings. 2023, ChemRxiv.

[62] Ziletti Angelo, Kumar Devinder, Scheffler Matthias, Ghiringhelli Luca M. Insightful classification of crystal structures using deep learning. Nat Commun 2018;9(1):2775.

[63] Huang Lei, Yu Weijiang, Ma Weitao, Zhong Weihong, Feng Zhangyin, Wang Haotian, Chen Qianglong, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. 2023, arXiv preprint arXiv:2311.05232.

[64] Chen Banghao, Zhang Zhaofeng, Langrené Nicolas, Zhu Shengxin. Unleashing the potential of prompt engineering in large language models: a comprehensive review. 2023, arXiv preprint arXiv:2310.14735.

[65] Wallace Eric, Xiao Kai, Leike Reimar, Weng Lilian, Heidecke Johannes, Beutel Alex. The instruction hierarchy: Training llms to prioritize privileged instructions. 2024, arXiv preprint arXiv:2404.13208.

[66] Li Xiang, Yao Yiqun, Jiang Xin, Fang Xuezhi, Meng Xuying, Fan Siqi, Han Peng, et al. Flm-101b: An open llm and how to train it with 100 k budget. 2023, arXiv preprint arXiv:2309.03852.